

Lab Assignment 3: Modules, Ports, and Data Types

Module: Verilog & Logic Synthesis

Objective:

To apply core Verilog-2001 constructs, port configurations, and memory modeling techniques. Proper implementation of tri-state logic, arrays, and dynamic data slicing is critical for accurately translating true hardware behavior.

Part 1: Advanced Theoretical Assignment

Answer the following questions comprehensively. Your responses must adhere strictly to Verilog-2001 standards.

Q1. Syntax Troubleshooting and Error Identification

Analyze the following Verilog code. Identify and explain all syntax, declaration, and structural errors present in the code.

```
module memory_ctrl (
    [7:0] data_in,
    output reg [0:15] addr_bus,
    inout data_bus,
    input wire clk
);

wire [31:0] full_word;
reg [7:0] grid_mem [0:3][0:1];
reg [4:0] start_idx;

always @(*) begin
    // Operation A
    grid_mem[0] = 8'hFF;

    // Operation B
    grid_mem[1][2:3] = 8'h00;

    // Operation C
    addr_bus[0:3] = full_word[start_idx + 3 : start_idx];
end
endmodule
```

Q2: Tri-State Logic and Bus Contention

You are designing a shared communication bus for a complex System-on-Chip (SoC) architecture.

- Explain **why** standard binary states (**0 and 1**) are **insufficient** for a **shared common data bus**.
- Write a module that has **inout port**. Using the **conditional operator**, write a **continuous assignment** to safely drive an **8-bit bus** named **shared_bus**. The bus should transmit **tx_data** when **tx_en** is high, and disconnect when low. **Simply, write a module of Tri-State buffer.**

Q3: Data Type Semantics

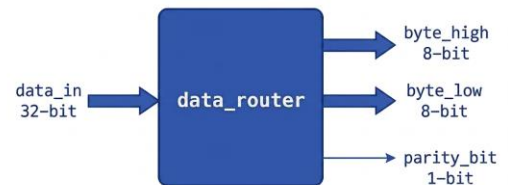
- Compare the behavior and application of **wire** and **reg** data types in Verilog. Specifically, address **where each can be legally assigned** and describe their **default initialization** behaviors during simulation.

Part 2: Practical Lab Exercise

Task 1: Data Router Design

Requirements:

- **Module Definition & Ports:** Create a module named **data_router**. It has
 - one 32-bit input named **data_in**.
 - three output ports:
 - **byte_high** (8-bit)
 - **byte_low** (8-bit)
 - **parity_bit** (1-bit)

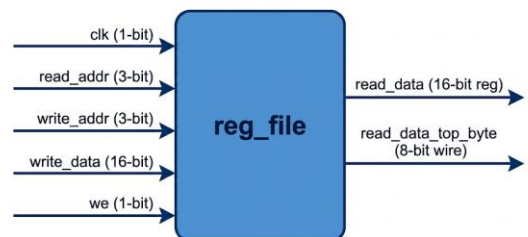


- Using **continuous assignment (assign)**, route the **lowest 8 bits of data_in** to **byte_low**.
- Inside an **always block**, assign the **highest 8 bits of data_in** to **byte_high**.
- Using **continuous assignment**, assign the **15th bit of data_in** to **parity_bit**.

Task 2: Register File Simulation

Requirements:

- **Module Definition & Ports:** Create a module named **reg_file**. It has
 - inputs:
 - **clk** (1-bit).
 - **read_addr** (3-bit).
 - **write_addr** (3-bit).
 - **write_data** (16-bit).
 - **we** (write enable, 1-bit).
 - output:
 - **read_data** (16-bit reg).
- A **memory (array)** named **register_file** that contains **8 elements**, where each element is a **16-bit vector**.
- **Synchronous write:** On the **positive edge of the clock**, if **we** is high, write the **write_data** into the memory element specified by **write_addr**.
- **Asynchronous read:** Continuously output the value of the memory element specified by **read_addr** to **read_data**.
- **Bonus Concept:** Additionally, create an output port **read_data_top_byte** (8-bit wire) and assign it the **top 8 bits** of the currently selected element of **read_data**.



Task 3: Design of a Configurable 2D Grid Memory Interface with Endian-Swapping

Objective:

Implement a memory module that utilizes 2-dimensional arrays, tri-state bidirectional buses, and variable part selects to manipulate data.

Requirements:

- **Module Definition & Parameters:** Create a module named **grid_mem_router**, it has:

- **A parameter:** **WORD_WIDTH** set to **32**.

- **inputs:**

- **clk:** 1-bit.
- **Rst:** 1-bit (Asynchronous active high)
- **oe:** 1-bit output enable.
- **endian_swap:** 1-bit control.
- **row_addr:** 2-bit.
- **col_addr:** 1-bit.

- **outputs:**

- **processing_word:** 32-bit.

- **bus_data:** 8-bit bidirectional (inout) wire vector representing the shared data bus.

- **Internal Storage:**

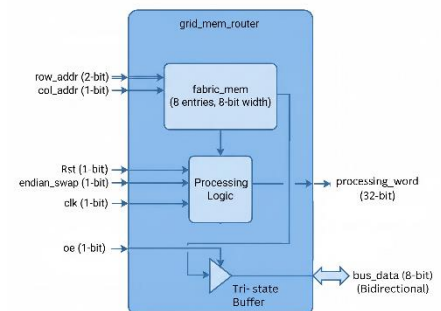
- Declare a **2-D** memory named **fabric_mem** that contains **4 rows** and **2 columns**. Each addressable location in this memory must hold **8 bits**.

- **Tri-State Output Logic:**

- When **oe** is **high**, the module should drive **bus_data** with the data currently stored in **fabric_mem** at the given **[row_addr] [col_addr]**.
- When **oe** is **low**, the output must be set to **high-impedance** to prevent electrical damage on the shared bus.

- **Slicing and Endian Swapping:**

- Create a **sequential procedural block** triggered by **positive clk** and cleared by **Asynchronous active high reset (Rst)**
- Inside the block, use an **integer i** in a **for loop (0 to 3)** to assemble the **processing_word** from the **memory**.
- If **endian_swap** is **0**, use the **variable part select +:** to map the **1st column** of the 2D memory blocks into **processing_word**.
- If **endian_swap** is **1**, use the **variable part select -:** to reverse the **byte order (Endianness swap)**, placing the **MSB** byte of into the **LSB** position of **processing_word**.



NOTE:

HOW TO WRITE A TESTBENCH:

- <https://vlsiverify.com/verilog/writing-a-testbench-in-verilog/>
- <https://chipverify.com/verilog/verilog-testbench>

Deliverables

Submit your **report** containing **all the answers** and your **.v files** containing all requested modules, alongside a **brief testbench (tb.v)** simulating the modules to verify correct operation.